

A Reverse Engineering Approach of Obfuscated Array

Wei Ding^{1,2}, ZhiMin Gu¹

1 School of Computer Science Technology, Beijing Institute of Technology, Beijing, China

2 College of Information Science and Engineering, Henan University of Technology, Zhengzhou, China

orangeding@163.com, dingwei@haut.edu.cn

Abstract—Recently, research community has advanced in type reconstruction technology for reverse engineering. However, emerging with obfuscated technology, data type reconstruction grows more and more difficult, and obfuscated code is easier to be monitored and analyzed by attacker or hacker. Therefore, it's essential to develop a novel approach to reverse engineering obfuscated array type based on refined CFG. We take split array for example and analyze feature in CFG and take advantage of compiler algorithm to identify obfuscated array.

Keywords—Obfuscated array, Reverse Engineering, Array Splitting, Array Folding, Array Flattening

I. INTRODUCTION

More and more research community is aware of importance and advances in the reconstruction of data structure have led to significant research, including well-known type inference algorithms Hindley-Milner[1], cartesian product[2], iterative analysis[3], abstract type inference[4].

The most common type reconstruction approaches are based on static analysis techniques in binary code, like IDA Pro[5], OllyDbg[6]. VSA[7] (Value Set Analysis) attempts to identify location of like-variable and evaluate possible value set, which use a-loc to find possible value set and track value of data object. ASI[8](Abstract Structure Identification) tries to statically partition array and variable in memory block according to memory access. It use system call and famous library function information types, the types of called parameters are known, which are marked with according types and propagate them. Then, Balakrishnan[9] combine VSA and ASI to identify simple structure, array and the nest

of array and structure. But static analysis method is difficult in basic aggregation structure.

REWAEDS[10] is a dynamic analysis method based on PIN analysis technology, which infer variable type by means of function parameters, return value and type signature instruction. In other words, it marks each location with timestamp type attribute and propagates it to other memory addresses, registers with program executed data flow, yet it can't deal with control flowing limited to executed path and

can't deal with obfuscated code. Howard [11] is complementary with Rewards, it is more powerful. It supplies assembler and debugger with data structure and type to relieve reverses engineering. It can reveal data structure layout according to memory access patterns and generates automatically debugger symbol. Howard can recover fields of aggregation structure, nested arrays, yet its results depend on runtime path coverage like any other dynamic analysis instruments.

TIE [12] develops a novel type inference system based on type reconstruction rules, which can be applied in static and dynamic analysis. The core of TIE is to infer type according how the codes use, for example, in arithmetic operation, SF flag is detected and it can infer two operands are signed int.

Laika[13] detects data syntactic structure through unsupervised learning during program execution, but accuracy of this technology is not enough for reverse engineer, it can identify part of obfuscated code with virus detectors, which is worthy for our reference.

Nevertheless, along with advance of data type reconstruction, a number of obfuscation techniques [14-16] are very effective against state-of-the-art disassembles, preventing a substantial fraction of a binary program from being disassembled correctly and reconstruct data structure.

For reverse engineering obfuscated array, two related questions are raised. The first question is how to layout array in the memory. The second is which technology is useful for understanding obfuscated array.

This paper presents a novel approach to reverse engineering obfuscated array type. We take split array and merging array for example and analyse feature in CFG and take advantage of compiler algorithm to identify obfuscated array. The remainder of this paper is organized as follows. Section 2 discusses array transformation. Section 3 discusses obfuscated array identification. Section 4 analyzes splitting array and how to use loop optimization to identify obfuscated array.

II. OBFUSCATED ARRAY TRANSFORMATION

Software obfuscation has several techniques such as computation transformation, layout transformation, ordering transformation and data transformation [15]. In this section we will discuss array obfuscation. As showed in [15], many

transformations can be used to obscuring operations performed on arrays in order to add the data complexity in the program and make a program much more difficult to reverse engineer.

A. Array Splitting

The obfuscation transformation can split an array into two (or more) new arrays. Collberg et al.[15] gives an example of an array. As shown in Fig 1. Array split transformation adopts index changes method. For example, S. Drape[18] presents a method to defining a split array by index change.

```

int A[10];
...
A[i]=...;
⇒
int A1[5];
int A2[5];
...
if((i%2)==0)
    A1[i/2]=...;
else
    A2[i/2]=...;

```

Fig. 1. Array splitting

B. Array Merging

Figure 2 shows arrays A and B are interleaved into a resulting array BC. The merging array elements from A and B are regularly distributed in resulting array.

```

int A[9],B[19];
A[i]=...;
B[i]=...;
⇒
int AB[29];
AB[3*i]=...;
AB[i/2*3+1+i%2]=...;

```

Fig. 2. Array merging

C. Array Folding

In [19], it uses homomorphism functions to array folding and gives a method to demonstrate how a one-dimensional array A can be folded into a two-dimensional array B.

```

int sum = 0;
int A[100] = {...};
for(i = 0; i < 100; i++)
    sum = sum + A[i];
⇒
int sum=0;
for(i=0;i<4;i++)
    for(j=0;j<25;j++)
        sum=sum+B[i,j];

```

Fig. 3. Array folding

D. Array Flattening

Array flattening is a reverse transformation to array folding, which flattens a two-dimensional array E into a one-dimensional array F.

```

int A[2,2];
for(i=0;i<=2;i++)
    for(j = 0; j <= 2; j++)
        swap(E[i,j],E[j,i]);
⇒
int A1[8]
for(i=0;i<=8;i++)
    swap(E[i],E[3*(i%3)+i/3]);

```

Fig. 4. Array flattening

III. OBFUSCATED ARRAY DETECTION

In order to reverse engineering obfuscated array, we present an algorithm to identify array based on control flow graph. Whatever array is transformed by obfuscation technique, it is used in similar ways by similar function and containing similar elements. According to it, we judge array how to transform.

A. Array Detection Method

Array detection is very important, especially for security. Array is a collection with the same type. In machine code, array is assigned as a sequence of memory location, denoted as $\{var \mid T(var)=type, R(var) \in [low,high]\}$, where $T(var)$ shows type of var, $R(var)$ expresses address of var and low and high in square brackets denotes upper and lower bound of array. Each element has the same operation behavior attributes and memory accessing mode in the memory domain. We can use it to reconstruct array.

In this paper, we consider three variant domains: 1) heap allocated memory, 2) stack allocated memory, 3) global allocated memory. The three memory accesses differ in base. Base of stack allocated memory is denoted as stack pointer, and the return value of allocated function such as new/delete represents base of heap allocated memory.

Array elements are accessed using index expression, which is constant numeric offset from base address. In the assembly code, array is expressed as $[base+index*scale+offset]$. For example, the i element address of array $int\ m[32]$ is $base+i*4$, which 4-byte denotes the size of the array element. Additionally, element of array has the same type, in other words, the index expression can not provide more information for type reconstruction, so we turn to $[base+offset]$. Howard[1] presents a dynamic array detection algorithm, which is divided into two generic schemes according to loops in real code implementation: one is relative to the previous element like $elem=(prev++)$, the other is relative to the base of an array, such as $elem=array[i]$. It recovers arrays when the program accesses them in loops. Although Howard can detect array, it can't pay attention on the size and basic type of array. To solve it, we initially construct refinement control flow graph to detect obfuscated array.

B. Construct Refinement Control Flow Graph

C Kruegel[20] et al. present novel disassembly process to improved obfuscated binaries based on refinement control flow graph. It is divided into three stages. Firstly, it partitions the program into functions that can be analyzed independently. Secondly, functions are divided into basic block and generate

coarse-grain control flow graph. At last, we gain specific control flow graph through refinement method.

For obfuscated array detection, we only take account of intra-procedural. Therefore, we act on the last two stages. Each address is taken as starting position of an instruction to disassemble, such that amount of assembler instructions are generated. C Kruegel et al. use heuristic algorithm to omit unreasonable instructions and gain real subset. Obviously, the flexibility of algorithm adopted heuristic method gets worse and is not fit for our purpose.

In this paper, a simple and intuitive method is adopted instead of heuristic method. We take advantage of method that C Kruegel presents to construct a coarse-grain control flow graph, and then refine this graph to drop unreasonable instructions. Basic block satisfy the following condition:

- 1) Each basic block contains a jump instruction at most.
- 2) The starting address of basic block is its previous instruction is jump instruction or at least a jump instruction can reach it or it is starting address of function.

Table 1. Refinement Control Flow Graph Algorithm

INPUTS: Binary File T
OUTPUT: Control Flow Graph
{ Each address is taken as starting position of an instruction to disassemble and gain superset S.
Partition S into B;
Generate a coarse control flow graph I;
Traverse I and find MaxIndegree(I) node N;
Gain the starting and ending address of N;
Judge whether N is not reasonable node;
if N can be reached by multiple other node
Tag N;
else drop unreasonable node;
Take starting address of N as ending address of predecessor;
Until all the nodes are tagged;
}

C. Loop Detection and Obfuscated Array Identification

The analysis of obfuscated array is very difficult because of diversity of array transformation. However, obfuscation is treated as inverse of compiler optimization, such that it can be partially removed by optimization algorithm.

As showed above, array detection depends on loop identification. Various transformations of array are closely related to loop transformation. CFG is denoted as $G(N,E,n_0)$, where N is node set of CFG, E is directed edge and n_0 is the first node. In CFG, a group of nodes satisfy the following condition, which is named as loop.

- 1) Having unique entry node. The entire path into any node of loop must pass through this entry node.
- 2) The group of nodes is strongly connected.

Loop identification is very important to detect array type. According to features of CFG, a node is found, which is

inevitable to other nodes of loop. In other words, it is an entry node. Then all the successor nodes are traversal recursively. When this node is traversal once again, it is loop node and we can deduce array size from loop count.

Table 2. Algorithm judged whether node is loop node

INPUTS: CFG $G(N,E,n_0)$
OUTPUT: Loop Node
{
$D(n_0)=\{n_0\}$;D(n0) is the entire nodes set passed through
for($n \in (N-\{n_0\})$)
$D(n)=N$;
Flag=true;
While(flag)
{
Flag=false;
For($n \in (N-\{n_0\})$)
{
$SUBSETD=\{n\} \cup (\cap S \in P(n)D(S));P(n)$ is predecessor sets
if($D(n) \neq SUBSETD$)
{flag=true;
$D(n)=SUBSETD$;
}
}
}
}

IV. OBFUSCATED ARRAY ANALYSIS AND REVERSE

ENGINEERING

In this section, we take split array for example and analyze feature in CFG and take advantage of compiler optimization algorithm to identify obfuscated array. Note that here said compiler algorithm refers to loop optimization.

	mov eax, [ebp+var_2C]
	add eax, 1
int a[10];	mov [ebp+var_2C], eax
int i;	cmp [ebp+var_2C], 9
for(i=0;i<=9;i++)	jg short loc_40104D
{	mov ecx, [ebp+var_2C]
a[i]=0;	mov [ebp+ecx*4+var_28], 0
}	jmp short loc_401031

Fig. 5. An unobfuscated program and assembly code. From assembly, we can judge loopcount is 9.

Array of Fig 5 can be spited into two array a1 and a2, which can be dissembled to Fig 6. According to Fig 6, we can obviously see splitting array a1 and a2 are in a loop. And array a1 and a2 are allocated sequence memory. Hence when loop satisfies the following condition, we use loop

optimization to deobfuscation.

arithmetic sequence.

V. CONCLUSION

In this paper, we present a novel approach to identify obfuscated array. Firstly, we refine CFG to detect loop and judge the size of array. On the other hand, we can remove obfuscation using compiler optimization. This paper focuses on loop optimization.

Our approach can be applied to transfer data obfuscation, however, it don't work well on reconstruction of complicated obfuscated data type, however, this approach is not accurate to abfuscated data type. In the future we can devote to research how to reconstruct splitting arrays, matrix and so on.

ACKNOWLEDGMENT

This work has been supported by Biological Hazard Monitoring Digital Technology of Grain Storage of the "863" national project 2012AA101608 and collaborative innovation center for modern grain circulation and safety.

REFERENCES

- 1) R. Milner. A theory of type polymorphism in programming. Journal of Computer and System Sciences, 17:348–375, 1978.
- 2) O. Agesen. The cartesian product algorithm: Simple and precise type inference of parametric polymorphism. In Proceedings of the 9th European Conference on Object-Oriented Programming (ECOOP'95), pages 2–26, London, UK, 1995.
- 3) C. Chambers and D. Ungar. Iterative type analysis and extended message splitting: Optimizing dynamically-typed object-oriented programs. In Proceedings of the SIGPLAN Conference on Programming Language Design and Implementation, pages 150–164, 1990.
- 4) P. J. Guo, J. H. Perkins, S. McCamant, and M. D. Ernst. Dynamic inference of abstract types. In Proceedings of the 2006 international symposium on Software testing and analysis (ISSTA'06), pages 255–265, Portland, Maine, USA, 2006. ACM.
- 5) (2005) DataRescue. High level constructs width IDA Pro. <http://www.hex-rays.com/idaipro/datastruct/datastruct.pdf>.
- 6) Ollydbg. <http://www.ollydbg.de/>.
- 7) G. Balakrishnan and T. Reps. Analyzing memory accesses in x86 binary executables. In Proc. Conf. on Compiler Construction (CC), April 2004.
- 8) G. Ramalingam, J. Field, and F. Tip. Aggregate structure identification and its application to program analysis. In Proceedings of the 26th ACM SIGPLAN-SIGACT symposium on Principles of Programming Languages, 1999.
- 9) Balakrishnan G, Reps T. Divine: Discovering variables in executables. Verification, Model Checking, and Abstract Interpretation. Springer Berlin Heidelberg, 2007: 1-28.
- 10) Z. Lin, X. Zhang, and D. Xu. Automatic reverse engineering of data structures from binary execution. In Proceedings of the 17th Annual Network and Distributed System Security Symposium (NDSS'10), San Diego, CA, March 2010.
- 11) Slowinska A, Stancescu T, Bos H. Howard: a dynamic excavator for reverse engineering data structures. Proceedings of NDSS. 2011.
- 12) Lee J, Avgerinos T, Brumley D. TIE: principled reverse engineering of types in binary programs. Proceedings of the 18th Network and Distributed System Security Symposium (NDSS). San Diego, USA: Internet society, 2011.
- 13) Cozzie A, Stratton F, Xue H, et al. Digging for data structures. Symposium on Operating Systems Design and Implementation (OSDI). 2008.

```
int a1[5],a2[5],i;
for(i=0;i<=9;i++)
{
    if((i%2)==0)
        a1[i%2]=0;
    else
        a2[i/2]=0;
}

.text:00401031 loc_401031:
.text:00401031      mov     eax, [ebp+var_2C]
.text:00401034      add     eax, 1
.text:00401037      mov     [ebp+var_2C], eax
.text:0040103A
.text:0040103A loc_40103A:
.text:0040103A      cmp     [ebp+var_2C], 9
.text:0040103E      jg      short loc_401080
.text:00401040      mov     ecx, [ebp+var_2C]
.text:00401043      and     ecx, 80000001h
.text:00401049      jns     short loc_401050
.text:0040104B      dec     ecx
.text:0040104C      or      ecx, 0FFFFFFFh
.text:0040104F      inc     ecx
.text:00401050
.text:00401050 loc_401050:
.text:00401050      test    ecx, ecx
.text:00401052      jnz     short loc_40106E
.text:00401054      mov     edx, [ebp+var_2C]
.text:00401057      and     edx, 80000001h
.text:0040105D      jns     short loc_401064
.text:0040105F      dec     edx
.text:00401060      or      edx, 0FFFFFFFh
.text:00401063      inc     edx
.text:00401064
.text:00401064 loc_401064:
.text:00401064      mov     [ebp+edx*4+var_14], 0
.text:0040106C      jmp     short loc_40107E
.text:0040106E ; -----
.text:0040106E
.text:0040106E loc_40106E:
.text:0040106E      mov     eax, [ebp+var_2C]
.text:00401071      cdq
.text:00401072      sub     eax, edx
.text:00401074      sar     eax, 1
.text:00401076      mov     [ebp+eax*4+var_28], 0
.text:0040107E
.text:0040107E loc_40107E:
.text:0040107E      jmp     short loc_401031
```

Fig. 6. An unobfuscated program and assembly code. From assembly, we can judge loopcount is 9.

- 1) When loop is executed, sequence memory domains are allocated.
- 2) During loop is executed, operator and accessed value of sequence memory are regularly changed in

- 14) C. Collberg and C. Thomborson. Watermarking, Tamper-Proofing, and Obfuscation - Tools for Software Protection. *IEEE Transactions on Software Engineering*, 28(8):735-746, August 2002.
- 15) C. Collberg, C. Thomborson, and D. Low. Taxonomy of Obfuscating Transformations. Technical Report 148, Department of Computer Science, University of Auckland, July 1997.
- 16) LINN, C., AND DEBRAY, S. Obfuscation of executable code to improve resistance to static disassembly. The 10th ACM Conference on Computer and Communications Security (CCS 2003), 2003.
- 17) Krishnamoorthy N, Debray S, Fligg K. Static detection of disassembly errors. WCRE'09. 16th Working Conference on. IEEE, 2009: 259-268.
- 18) Drape S. Generalising the array split obfuscation. *Information Sciences*, 2007, 177(1): 202-219.
- 19) Zhu W, Thomborson C D, Wang F Y. Obfuscate arrays by homomorphic functions. *GrC*. 2006: 770-773.
- 20) Kruegel C, Robertson W K, Valeur F, et al. Static Disassembly of Obfuscated Binaries. *USENIX security Symposium*. 2004, 13: 18-18.

Wei Ding was born in Anhui province, china. After graduated from Henan Institute of Technology in 2002, she entered into Zhengzhou University, and gained Master Degree. Then she worked into Henan Universality of Technology and became a doctoral student of Beijing Institute of Technology in 2009. Her research area of interest is binary analysis and software security.